# AI SDLC Copilot

Team Handbook for Business Analysts, Developers and QA

FY27 NFTS Code-a-thon · Use Case 2 · 05 November 2026

For Business Analysts, Developers and QA engineers who need to work on the project by hand: what each file does, how the agents work, and exactly where to make a change.

Repository: https://github.com/akyapada/ai-sdlc-copilot (private; ask the owner for an invite)

Hosted demo: https://ai-sdlc-copilot.onrender.com (free plan; sleeps when idle, data resets on redeploy)

## Contents


1. Quick start
2. How the workflow fits together
3. Project map
4. The agents
5. Where to make a change
6. Business Analyst guide
7. QA guide
8. Developer guide
9. Troubleshooting

For Business Analysts, Developers and QA engineers who need to work on the project by hand: what each file does, how the agents work, and exactly where to make a change.

Repository: https://github.com/akyapada/ai-sdlc-copilot (private; ask the owner for an invite) Hosted demo: https://ai-sdlc-copilot.onrender.com (free plan; sleeps when idle, data resets on redeploy)

# 1. Quick start

Requires Node.js 22.13 or newer (Node 24 recommended) and Git. On Windows PowerShell 5.1, run commands one per line; `&&` does not work there.

```
git clone https://github.com/akyapada/ai-sdlc-copilot.git
cd ai-sdlc-copilot
npm install
copy .env.example .env        (macOS/Linux: cp .env.example .env)
npm start                     -> http://127.0.0.1:3000
```

Demo accounts: `admin@demo.local`, `pm@demo.local`, `architect@demo.local`, `dev@demo.local`, `qa@demo.local`. The password is the `DEMO_PASSWORD` value in `.env` (on Render: the value set in the service's Environment settings). Accounts are created only the first time the database is created; to reset everything, stop the app and delete `data/sdlc.db`.

Without Azure OpenAI settings the app uses the **offline engine**: rule-based versions of every agent, so the full workflow works with no network. Set `AZURE_OPENAI_ENDPOINT` and `AZURE_OPENAI_API_KEY` in `.env` to use the bootcamp `gpt-5-5` deployment.

| Command | What it does |
|---|---|
| `npm start` | Run the app on port 3000 (`PORT` changes it) |
| `npm run dev` | Run with auto-restart on file changes |
| `npm test` | API, agent unit and data-integrity tests (64 tests, about 10 s) |
| `npm run test:e2e` | Playwright browser tests in headless Microsoft Edge (9 tests) |
| `npm run load` | Load test with latency budgets (fails if a budget is missed) |
| `npm run test:all` | All three, in order |

# 2. How the workflow fits together

| Step | Stage | Who approves | Output |
|---|---|---|---|
| 0 | Upload `.docx` + `.xlsx` | – (PM, Architect or Admin uploads) | Validated, hashed, versioned files |
| 1 | Gate 1 · Requirements | PM or Architect | FR/NFR, conflicts, ambiguities, gaps, risks |
| 2 | Gate 2 · Architecture | Architect only | Components, interactions, integrations, security |
| 3 | Gate 3 · Dev tasks | PM | Sequenced, sized, assigned tasks |
| 4 | Gate 4 · QA tests | QA | Test cases of 8 types, traced to requirements and tasks |
| 5 | Deliver | – (PM or Admin pushes) | Excel/CSV/Markdown export, ADO or Jira work items (approved versions only) |

At every gate the same loop runs: the **orchestrator** checks the gate is open, the **guardrail** redacts sensitive data and quarantines prompt injection, **retrieval** finds source evidence, a **specialist** drafts the artefact, the **planner** assigns work, the **critic** scores it, and the result is saved as a new version **in review**. A person then approves, rejects (reason required), comments, edits inline, or asks the **review-feedback agent** to regenerate. A later stage cannot be generated until the one before it is approved; any upstream change marks later stages **stale** and blocks their approval.

## Who can do what (role × stage)

| Action | Admin | PM | Architect | Developer | QA |
|---|---|---|---|---|---|
| Create projects, manage members | Yes | Yes | – | – | – |
| Upload requirements / team files | Yes | Yes | Yes | – | – |
| Requirements: generate / edit / approve | gen, edit | all | all | read | read |
| Architecture: generate / edit / approve | gen, edit | gen | all | read | read |
| Dev tasks: generate / edit / approve | gen, edit | all | gen | read | read |
| QA tests: generate / edit / approve | gen, edit | gen | read | read | all |
| Export files | Yes | Yes | Yes | – | Yes |
| Configure / push ADO and Jira | Yes | Yes | – | – | – |
| Users, global audit log | Yes | – | – | – | – |

Admins can generate and edit any stage but can never approve one. There is no separate BA role in the app: a Business Analyst normally signs in with a **PM** account (or registers as PM and is activated by an admin). The matrix lives in `src/security.js` → `STAGE_PERMISSIONS` (see section 5).

# 3. Project map

```
ai-sdlc-copilot/
  src/
    server.js              Express app: security headers, routes, static UI, error handler
    config.js              Reads .env; production start-up guards; host/port/proxy settings
    db.js                  SQLite schema (users, projects, files, artifacts, reviews, audit...), demo users
    security.js            Sessions, CSRF guard, audit(), role x stage permission matrix, token encryption
    files.js               Upload validation and parsing of .docx (requirements) and .xlsx (team)
    llm.js                 Azure OpenAI call (JSON mode, timeout, untrusted-input wrapper)
    agents/
      orchestrator.js      Workflow, gates, versions, stale detection, approvals, traceability
      specialists.js       AI prompts (ROLES), output schemas (SCHEMAS), normalize()
      mock.js              Offline engine: rule-based analyst, architect, planner, QA designer
      retrieval.js         BM25 index of the document; evidence for every item
      critic.js            Rule checks + optional AI review; 0-100 score
      guardrail.js         PII/secret redaction, prompt-injection quarantine, unsupported claims
      feedback.js          Review-feedback agent (commands or AI revision) + version diff
      assign.js            Skill/capacity assignment, sequencing, cycle detection
    routes/
      auth.js              Register, login, lockout, logout, change password
      admin.js             Users, activation, roles, password reset, global audit
      projects.js          Projects, members, uploads, stages, decisions, exports, integrations
    exporters/
      bundle.js            Excel workbook, Jira CSV, ADO CSV, Markdown, JSON
      integrations.js      Azure DevOps and Jira push (dry run, idempotent, SSRF guard)
  public/
    index.html, styles.css Single-page shell and styling
    app.js                 The whole UI: views, stage tables (SPECS), editing, review panel
  test/
    api.test.js            End-to-end API tests incl. the panel stress pack
    agents.test.js         Unit tests per agent, natural-language feedback edge cases
    integrity.test.js      Concurrency, immutable history, isolation, lockout, audit coverage
    helpers.js             Boots the app in-memory for tests
  e2e/
    workflow.spec.js       Playwright browser tests of the main journeys
    seed.spec.ts           Seed for the Playwright test agents (signs in as PM)
  specs/                   Test plans written by the Playwright planner agent
  .claude/agents/          Playwright planner / generator / healer agent definitions
  .mcp.json                Starts the Playwright test MCP server for those agents
  scripts/load-test.mjs    Load test
  samples/                 Handbook sample inputs; panel-pack/ = 10 hostile test files
  docs/                    This handbook, QA-Test-Catalog.xlsx
  render.yaml              Render hosting blueprint
  playwright.config.js     Browser test settings (Edge, port 3100, throwaway data folder)
  .env.example             Every setting with comments (copy to .env)
```

Never committed (see `.gitignore`): `.env`, `data/` (database and uploads), `node_modules/`, test reports.

# 4. The agents

| Agent | File → main function | What it does | AI mode / offline |
|---|---|---|---|
| Orchestrator | `orchestrator.js` → `runStage`, `regenerateWithFeedback`, `decide` | Enforces gates, builds context from approved upstream versions, runs the other agents, saves versions | Same in both |
| Requirements analyst | `specialists.js` → `generate('requirements')`; offline `mock.js` → `analyzeRequirements` | FR/NFR, conflicts, ambiguities, boundaries, gaps, dependencies, risks, assumptions | Prompt `ROLES.requirements` / rule-based parser |
| Architect | `generate('architecture')`; `mock.js` → `draftArchitecture`, `techStack` | Components, interactions, integrations, data flows, tech choices from team skills, security | Prompt / rules |
| Delivery planner | `generate('tasks')`; `mock.js` → `planTasks`; `assign.js` | Tasks, dependencies, sequence, size, hours; skill- and capacity-based assignment | Prompt + deterministic assignment |
| QA designer | `generate('tests')`; `mock.js` → `generateTests` | Positive, negative, boundary, integration, regression, security, usability, performance tests | Prompt / rules |
| Retrieval | `retrieval.js` → `buildIndex`, `search`, `collectEvidence` | Chunks the document, adds evidence (the paperclip icon) to items, grounds prompts | Same in both |
| Critic | `critic.js` → `deterministicReview`, `review` | Coverage, conflicts, cycles, capacity, missing test types; score; triggers one self-revision below the threshold (AI mode) | Rules + optional AI review |
| Guardrail | `guardrail.js` → `sanitizeDocument`, `scanText`, `reviewOutput` | Redacts cards/keys/emails/phones before any agent sees the document, quarantines injection paragraphs, flags unsupported claims | Same in both |
| Review-feedback | `feedback.js` → `reviseWithFeedback`, `diffContent` | New version from reviewer suggestions with change log and diff | Free text (AI) / commands (offline) |

**Data contract.** Every stage's output is a JSON object whose shape is defined twice: in `SCHEMAS` (what the AI is asked to return) and in `normalize()` (what the code guarantees). The UI (`public/app.js` → `SPECS`), the critic, exports and integrations all read that shape. When you add or rename a field, change it in all of those places (recipe 5.2).

# 5. Where to make a change

## 5.1 Quick reference

| I want to… | Change this | Notes |
|---|---|---|
| Change what the AI is told to do at a stage | `src/agents/specialists.js` → `ROLES.<stage>` | Plain-English instructions. Keep "reference requirement IDs" rules, or the guardrail flags unsupported claims |
| Change the AI's output format | `specialists.js` → `SCHEMAS.<stage>` and `normalize()` | Then follow recipe 5.2 |
| Change the critic's AI review instructions | `src/agents/critic.js` → `review()` system prompt | Rule checks are in `deterministicReview()` |
| Change how strict the critic is | `critic.js` → `score()` weights (high 12, medium 5, low 1); `REQUIRED_TEST_TYPES` | Verdict bands: ≥85 ready, ≥60 caution |
| Change when the AI self-revises | `.env` → `CRITIC_SELF_REVISE_THRESHOLD` (default 70) | AI mode only |
| Change who may generate / edit / approve | `src/security.js` → `STAGE_PERMISSIONS` | Admin can generate and edit but never approve (`can()`) |
| Recognise more vague words | `src/agents/mock.js` → `VAGUE` | Key = word or phrase, value = the clarifying question |
| Add a new gap rule | `mock.js` → `analyzeRequirements`, the `addGap(...)` lines | Pattern: `if (!/keyword/i.test(text)) addGap('GENERAL', 'what is missing', 'recommendation')` |
| Detect new boundary phrases | `mock.js` → `boundaries()` | e.g. add "at most" / "no more than" |
| Detect new conflicts | `mock.js` → the `addConflict(...)` block | Conflicts block approval until resolved or acknowledged |
| Change suggested tech stack | `mock.js` → `techStack()` | Driven by team-sheet skills |
| Change how work is assigned | `src/agents/assign.js` → `SKILL_CATEGORIES`, `QA_PREFERENCE`, score in `assignTasks()` | Skill fit dominates; load spreads work; 0 capacity = never assigned |
| Add a sensitive-data pattern | `src/agents/guardrail.js` → `PATTERNS` | Each: `{ type, severity, re }`; matches are redacted before the AI |
| Add a prompt-injection phrase | `guardrail.js` → `INJECTION` | Matching paragraphs are quarantined |
| Add a policy rule (e.g. "store passwords") | `guardrail.js` → `POLICY_TERMS` | Shown in the Governance panel |
| Change team-sheet columns / rules | `src/files.js` → `parseTeamSheet()`, `validateTeam()`, `KNOWN_TEAM_ROLES` | Required: Name, Role, Skills, Capacity (0–80) |
| Change requirements-document checks | `files.js` → `validateRequirements()` | Errors block the upload; warnings are shown |
| Change upload size limit | `.env` → `MAX_UPLOAD_MB` (default 10) | Zip-bomb limit: `MAX_UNCOMPRESSED` in `files.js` |
| Add a feedback-box command | `src/agents/feedback.js` → `ruleBasedRevision()` | Add a regex branch before the "reviewer directive" fallback; reject with `rejected(line, why)` |
| Change allowed priority values | `feedback.js` → `PRIORITIES` and `public/app.js` → `SPECS` select `options` | Keep both in sync |
| Show / hide / reorder UI columns | `public/app.js` → `SPECS.<stage>.sections[].cols` | Types: text, longtext, list, refs, select, number, assignee, id, ro |
| Change Excel / CSV export columns | `src/exporters/bundle.js` → `toExcel()`, `toJiraCsv()`, `toAdoCsv()` | CSV cells are formula-escaped by `csvCell()` |
| Change ADO work item / Jira issue mapping | UI fields on Export page, or `src/exporters/integrations.js` → `adoPlan()`, `jiraPlan()` | Only approved versions are pushed |

| I want to... | Change this | Notes |
|---|---|---|
| Change lockout or session length | `src/routes/auth.js` → `LOCK_AFTER`, `LOCK_MINUTES`; `src/security.js` → `SESSION_HOURS` | |
| Use a different AI deployment / model | `.env` → `AZURE_OPENAI_*`; request body in `src/llm.js` → `chatJSON()` | `AI_MODE=mock` forces the offline engine |

## 5.2 Recipe: add a field to an artefact (example: "Business value" on functional requirements)

1. `specialists.js` → `SCHEMAS.requirements`: add `"businessValue":"High|Medium|Low"` to the functional item.
2. `specialists.js` → `normalize()` requirements branch: add `businessValue: r.businessValue || 'Medium'` to the functional map, so old versions and the offline engine still produce the field.
3. Offline engine (optional): `mock.js` → `analyzeRequirements`, set `businessValue` when building `functional`.
4. UI: `public/app.js` → `SPECS.requirements` → functional `cols`: add `S('businessValue', 'Value', 'select', { options: ['High', 'Medium', 'Low'] })`.
5. Export: `bundle.js` → `toExcel()` Requirements sheet columns: add `['Business value', 'businessValue', 14]`.
6. Test: add an assertion in `test/agents.test.js` or `test/api.test.js`; run `npm test`.

## 5.3 Recipe: change who approves a stage (example: let a BA-type PM approve architecture too)

1. `src/security.js` → `STAGE_PERMISSIONS.architecture.approve`: `['ARCHITECT', 'PM']`.
2. `public/app.js` → `renderSide()`: update the "Only the Software Architect can approve" text.
3. `test/api.test.js` → the test asserting "PM cannot approve architecture": update the expectation.
4. Note the change in the README's governance section; this is a governance decision, so agree it with the team first.

## 5.4 Recipe: add a new user role (example: BA)

1. `src/db.js`: add `'BA'` to the `role` CHECK constraint and to `ROLES`. SQLite cannot change a CHECK constraint in place, so delete `data/sdlc.db` in development (or write a migration).
2. `src/security.js`: add `'BA'` to the relevant `STAGE_PERMISSIONS` lists and to `stagePermissions()` (upload, export).
3. `public/app.js`: add the option to the Register form and the Admin role select (`roles` array in `viewAdmin`).
4. `src/routes/auth.js` → register: allowed roles come from `ROLES` (ADMIN is always refused).
5. Add tests for the new permissions.

## 5.5 Coding rules that keep the app safe

- Every state-changing route calls `audit(req, ACTION, {...})`. Add one for anything new.
- Every project route goes through `projectAccess`; every stage action through `needStage('generate'|'edit')` or `can()`.
- Never trust uploaded or AI text: render with `esc()` in the UI, keep it inside `<document>`/`<input>` tags in prompts.
- Never overwrite a version: create a new one via `finalize()` in the orchestrator.
- Keep the offline engine working; demos and tests depend on it.

# 6. Business Analyst guide

## 6.1 Writing a requirements document the agents read well

The offline engine parses structure; the AI engine copes with anything, but the same structure gives it better traceability.

| Write it like this | Why |
|---|---|
| `Project: GlobalCommerce Express Checkout` as the first line | Becomes the project title |
| Numbered section headings: `2. Functional Requirements` | Ends the previous requirement |
| One requirement per heading line: `FR-101: Single-Click Order Placement` | IDs FR-, NFR-, BR-, REQ-, UR-, SR- are kept as-is; NFR- becomes non-functional |
| Labels on their own line: `Description:`, `User Flow:`, `System Behavior:`, `Acceptance Criteria:` | Lines under `Acceptance Criteria:` become testable criteria (one test each) |
| `Out of Scope:` … and `System Constraint:` … lines in their own section | Feed scope-conflict detection and constraint boundary tests |
| Numbers with units: "less than 200 ms", "5,000 requests per second", "99.9%" | Become NFR metrics and performance tests |
| Explicit boundaries: "a score of 30 or more", "up to and including $10,000" | Avoids ambiguity questions about inclusive / exclusive limits |
| Unique IDs, and references only to IDs that exist | Duplicates and dangling references are flagged |

Avoid: vague words (fast, brief, secure, user-friendly, as needed, appropriate), real personal data or keys (they are redacted and flagged), and text addressed to the AI (it is quarantined).

## 6.2 Team template (Excel)

Sheet named `Team` (or the first sheet), header row: **Name, Role, Skills, Capacity**, optional **Email**. Skills separated by commas or semicolons, for example `C#, ASP.NET Core, SQL`. Capacity is hours per week, 0–80; 0 means unavailable. Roles recognised: Developer, QA, Architect, PM, BA, Designer, DevOps, Tester, Lead. Only Developer/DevOps/Lead/Engineer rows get dev tasks; QA/Tester rows get test cases. No formulas, no duplicate names.

## 6.3 Reviewing Gate 1

1. Read the **Conflicts** table first: approval is blocked until each is resolved or explicitly acknowledged with a comment.
2. Work through **Ambiguities** with the product owner, then record answers with the feedback box: `resolve A-05: a score of exactly 30 requires OTP`.
3. Check **Gaps** and **Risks**; add missing ones: `add risk: FraudGuard SLA unknown for FR-102`.
4. Hover the paperclip icon to see the source text behind an item.
5. Approve with a comment. Your name, the time and the version are recorded.

## 6.4 Feedback-box commands (work offline; AI mode also accepts free text)

| Command | Stages | Example |
|---|---|---|
| `resolve <ID>: <answer>` | Requirements (A-, X- items) | `resolve X-01: video moves to Phase 2` |
| `add risk / assumption / gap / question: <text>` | Requirements | `add assumption: OTP valid for 5 minutes` |
| `add component / security control / question / assumption: <text>` | Architecture | `add component: Feature flag service for FR-101` |

| Command | Stages | Example |
|---|---|---|
| `use <new> instead of <old>` | Any | `use PostgreSQL instead of Azure SQL` |
| `add task: <text>` | Dev tasks | `add task: Feature toggle for FR-101` |
| `add test: <type> - <text>` | QA tests | `add test: Boundary - exactly 30 score for FR-102` |
| `assign <ID> to <name>` | Dev tasks, QA tests | `assign T-014 to Asha Rao` |
| `priority of <ID> to <value>` | Requirements (Must/Should/Could), tasks (P1–P3), tests (High/Medium/Low) | `priority of T-022 to P1` |
| `estimate <ID> to <hours>` | Dev tasks | `estimate T-017 to 24` |
| `rename <ID> to <text>` | Any | `rename C-04 to Checkout Orchestrator` |
| `remove <ID>` | Any | `remove TC-006` |

Separate several commands with new lines or `;`. Commands that cannot be applied (unknown ID, ambiguous name, invalid value) are listed as **Not applied** with the reason. Anything else is kept as a reviewer directive for the next reviewer. Tick "Also fix the critic's findings" to auto-add missing tasks or tests for uncovered requirements.

## 6.5 Sharing results

Export page: Excel workbook (all stages, traceability, approvals), Markdown report, Jira or ADO CSV for manual import, or direct push to ADO/Jira (PM). Tick "Approved versions only" for anything that leaves the team.

# 7. QA guide

## 7.1 Test suites

| Suite | File | Run | Covers |
|---|---|---|---|
| Agent unit | `test/agents.test.js` | `npm test` | Each agent; 17 natural-language feedback edge cases |
| Data integrity | `test/integrity.test.js` | `npm test` | Concurrent generation/approval, immutable history, isolation, lockout, audit |
| API end-to-end | `test/api.test.js` | `npm test` | Whole workflow and panel stress pack |
| Browser | `e2e/workflow.spec.js` | `npm run test:e2e` | Real clicks in headless Edge; report in `e2e-report/index.html` |
| Load | `scripts/load-test.mjs` | `npm run load` | 20 projects end to end + 200 reads from 50 users, p95 budgets |
| Test-case catalog | `docs/QA-Test-Catalog.xlsx` | Excel | 70 cases mapped to automated tests; Status column drives the Summary |
| Stress inputs | `samples/panel-pack/` | Upload by hand | 10 hostile files; expected results in `PANEL-SCENARIO.md` |

Browser tests use your installed Microsoft Edge. To use Chrome: `set E2E_CHANNEL=chrome` (PowerShell: `$env:E2E_CHANNEL='chrome'`). For Playwright's own Chromium: `npx playwright install chromium` and set `E2E_CHANNEL` to an empty value.

## 7.2 Adding a test by hand

- **API or agent test:** copy a block in `test/api.test.js` or `test/agents.test.js`. `helpers.js` gives you `startApp()`, `client()`, `setupProject()` (logged-in users for every role plus a project with the sample files).
- **Browser test:** add a `test(...)` to `e2e/workflow.spec.js`. Tests run in order and share one server; sign in with the `login(page, 'pm')` helper. The E2E server runs on port 3100 with its own empty data folder and password `E2ePassword123`, so it never touches your local data.
- **Catalog:** add a row to `docs/QA-Test-Catalog.xlsx` with the test's file and exact name.

## 7.3 Playwright test agents: planner, generator, healer

The repo includes Playwright's three AI test agents, set up with `npx playwright init-agents --loop=claude` (Playwright 1.64). They run inside **Claude Code** and drive a real browser through the Playwright test MCP server (`.mcp.json`).

| Agent | File | Input → output |
|---|---|---|
| **Planner** | `.claude/agents/playwright-test-planner.md` | Explores the running app from the seed state → Markdown test plan in `specs/` |
| **Generator** | `.claude/agents/playwright-test-generator.md` | A plan in `specs/` → executable tests in `e2e/` (verifies selectors live while writing) |
| **Healer** | `.claude/agents/playwright-test-healer.md` | A failing test → replays it, inspects the page, fixes locators/waits, re-runs until it passes, or marks it `test.fixme()` with an explanation if the app itself is broken |

**Seed.** `e2e/seed.spec.ts` signs in as the PM and lands on Projects; every plan and generated test starts from there. Change it if you want agents to start elsewhere (for example inside a project with files uploaded).

**How to use them**

1. Open a terminal in the repo and start Claude Code (`claude`). Approve the `playwright-test` MCP server when asked. The server uses `playwright.config.js`, which starts the app on port 3100 automatically.
2. **Plan:** "Use the playwright-test-planner agent to create a test plan for the approval gates and the review-feedback box. Save it as specs/approval-gates.md."
3. **Review the plan** (QA judgment): delete duplicates, add the edge cases from section 6.4 and the panel pack.

4. **Generate:** "Use the playwright-test-generator agent to generate tests for every scenario in specs/approval-gates.md."
5. Run `npm run test:e2e`.
6. **Heal:** when a UI change breaks tests: "Use the playwright-test-healer agent to fix the failing tests in e2e/."
7. Review the diff before committing: the healer may change assertions, and a "fixed" test must still check what the requirement says.

Notes: `.mcp.json` uses `cmd /c npx …` (Windows). On macOS/Linux change it to `"command": "npx", "args": ["playwright", "run-test-mcp-server"]`. To regenerate the agent files after a Playwright upgrade, run `npx playwright init-agents --loop=claude` again (it rewrites the three agent files; check that `e2e/seed.spec.ts` still signs in afterwards). Other loops: `--loop=vscode` (VS Code Copilot), `--loop=codex`, `--loop=opencode`.

## 7.4 Before a demo (manual checklist)

1. `npm run test:all` is green.
2. Hosted site: open it 5 minutes early (free plan wakes in about a minute); sign in as each role once.
3. Upload files 5–10 from the panel pack: each is rejected with a clear reason.
4. Upload files 1 and 2: Conflicts table shows X-01..X-03; Governance panel shows QUARANTINED, CARD_NUMBER, API_KEY.
5. Approve is blocked; `resolve X-01: …` works; approval with acknowledgement and comment succeeds.
6. Architecture diagram renders; tasks show skill-gap warning; ADO dry run lists work items.
7. Update the Status column in the QA catalog.

# 8. Developer guide

## 8.1 Request flow

```
Browser (public/app.js)  --fetch /api/... + X-Requested-With header-->  server.js
  -> csrfGuard -> authenticate (JWT cookie) -> projectAccess (membership) -> route in routes/projects.js
  -> permission check (needStage / can) -> orchestrator (runStage / decide / ...) -> agents
  -> db.js (SQLite) + audit_log row  -> JSON response -> UI re-renders the stage
```

## 8.2 Database tables (`src/db.js`)

| Table | Holds |
|---|---|
| `users` | Accounts, role, active flag, lockout counters |
| `projects`, `project_members` | Projects and who can see them |
| `files` | Every upload version: SHA-256, parsed JSON, validation result |
| `artifacts` | Every version of every stage: content, critic, guardrail, evidence, diff, upstream versions, status |
| `reviews` | Approve / reject / comment / regenerate decisions with reviewer and time |
| `audit_log` | Every action (also denials and rejected uploads) |
| `integrations`, `sync_items` | ADO/Jira settings (token AES-256-GCM encrypted) and what was already pushed |

## 8.3 Settings (`.env`, all optional locally)

| Variable | Default | Meaning |
|---|---|---|
| `PORT` / `HOST` | 3000 / 127.0.0.1 | Listen address (Render: 0.0.0.0 automatically) |
| `APP_SECRET` | random per start | Signs sessions, encrypts tokens; required in production (32+ chars) |
| `DEMO_PASSWORD` / `SEED_DEMO_USERS` | Bootcamp@2026 / true | Demo accounts; production refuses the default |
| `AZURE_OPENAI_ENDPOINT`, `_API_KEY`, `_DEPLOYMENT`, `_API_VERSION` | –, –, gpt-5-5, 2024-10-21 | AI engine |
| `AI_MODE` | auto | `auto` = AI if configured, `mock` = always offline |
| `CRITIC_SELF_REVISE_THRESHOLD` | 70 | Critic score that triggers one AI self-revision |
| `MAX_UPLOAD_MB` | 10 | Upload size limit |
| `ALLOW_PRIVATE_INTEGRATION_HOSTS` | false | Allow on-prem Jira on private IPs |
| `TRUST_PROXY` | loopback (Render: 1) | Reverse proxies in front of the app |

## 8.4 Git and deployment

- Work on a branch, open a pull request, merge to `main`. Every push to `main` redeploys the hosted site automatically (Render reads `render.yaml`).
- Hosted data (projects, uploads) resets on every redeploy on the free plan; don't push just before a demo.
- Secrets live only in `.env` (local) or Render → Environment (hosted). Never commit them; `.gitignore` already excludes `.env` and `data/`.
- Production refuses to start without a strong `APP_SECRET` and a non-default `DEMO_PASSWORD`; the Render log says which.

# 9. Troubleshooting

| Problem | Fix |
|---|---|
| `The token '&&' is not a valid statement separator` | Windows PowerShell 5.1: run each command on its own line, or use `;` |
| `EADDRINUSE` on port 3000 | Another copy is running; stop it or set `PORT=3001` |
| Sign-in fails with the right password | The database was created with an older `DEMO_PASSWORD`; delete `data/sdlc.db` (local) or reset the password as admin |
| "Account locked" | 5 failed attempts lock for 15 minutes; an admin can click **Unlock** |
| Stage button is disabled | Hover it: the upstream stage is not approved yet, or a file is missing |
| "Upstream inputs changed" on approve | The version is stale; regenerate the stage first |
| Approve says "unresolved conflicts" | Resolve via the feedback box or tick acknowledge and add a comment |
| Engine badge says offline-mock | Azure OpenAI settings are missing or wrong; check `.env` and the server log |
| Render log: "Refusing to start in production" | Set `APP_SECRET` (32+) and `DEMO_PASSWORD` (12+, not the default) in Render → Environment |
| Hosted site takes a minute to load | Free plan was asleep; it wakes on the first request |
| Browser tests can't find Edge | Set `E2E_CHANNEL=chrome`, or install Playwright's Chromium (section 7.1) |
| Playwright agents don't appear in Claude Code | Start `claude` from the repo root; approve the `playwright-test` MCP server; on macOS/Linux fix `.mcp.json` (section 7.3) |